Django Complete Notes (A–Z)

Django Notes – All Topics

1. Django Migrations
2. Django File Management
3. Testing in Django
4. Django User Authentication
5. Django Cache Framework
6. Conditional View Processing
7. Composite Primary Keys
8. Cryptographic Signing
9. Sending Email
10. Internationalization and Localization
11. Logging
12. Pagination
13. Django Security
14. Performance and Optimization
15. Serializing Django Objects
16. Django Settings
17. Django Signals
18. Django System Check Framework
19. External Django Packages
20. Django Asynchronous Support
21. Django Tasks Framework

Additional Sections

22. Common Django Commands
23. Production Deployment Checklist
24. Django Interview Quick Revision
25. Recommended Learning Order

Part 1: Django Migrations

1. What Are Django Migrations?

Django migrations are used to propagate changes made to Django models into the database schema.

Whenever we create, modify, or delete a model or field, migrations help apply those changes to the database.

Examples of Model Changes

- Creating a new model.
- Adding a new field.
- Removing an existing field.
- Changing a field's type or options.
- Deleting a model.

2. Common Migration Commands

Create Migration Files

```bash
python manage.py makemigrations
```

This command detects changes in models and creates migration files.

Apply Migrations

```bash
python manage.py migrate
```

This command applies unapplied migrations to the database.

Show Migration Status

```bash
python manage.py showmigrations
```

This command displays migrations and indicates which ones have been applied.

Display SQL for a Migration

```bash
python manage.py sqlmigrate app_name 0001
```

This command displays the SQL statements Django would execute for the

````
specified migration.

### Check Migration Changes

```bash
python manage.py makemigrations --check
```

This command exits with a non-zero status if model changes are not
reflected in migration files.

## 3. The Migration Workflow

```text
Modify models.py
       |
       v
makemigrations
       |
       v
Migration files created
       |
       v
migrate
       |
       v
Database schema updated
```

## 4. Migration Files

Migration files are stored in an app's `migrations/` directory.

Example:

```text
myapp/
    migrations/
        __init__.py
        0001_initial.py
        0002_add_email.py
````
````

````
    models.py
```

Migration files describe database operations such as creating tables, adding fields, and modifying relationships.

## 5. Migration Dependencies

A migration may depend on another migration.

Django uses dependencies to determine the correct execution order.

```python
class Migration(migrations.Migration):

    dependencies = [
        ("users", "0001_initial"),
    ]
```

This migration depends on the initial migration of the `users` app.

## 6. Reversing Migrations

To reverse an app to a previous migration:

```bash
python manage.py migrate app_name 0001
```

To reverse all migrations for an app:

```bash
python manage.py migrate app_name zero
```

**Warning:** Reversing migrations may delete tables or data. Review the migration operations before executing the command.

## 7. Data Migrations
````
````

Data migrations modify existing database data rather than only changing the schema.

Create an empty migration:

```bash
python manage.py makemigrations --empty app_name
```

Example:

```python
from django.db import migrations


def update_users(apps, schema_editor):
    User = apps.get_model("users", "User")

    User.objects.filter(is_active=True).update(
        username="active_user"
    )


class Migration(migrations.Migration):

    dependencies = [
        ("users", "0001_initial"),
    ]

    operations = [
        migrations.RunPython(update_users),
    ]
```

In real applications, ensure that data updates do not unintentionally assign the same unique value to multiple records. Use historical models through `apps.get_model()` rather than importing current models directly.

8. Historical Models

Historical models represent a model's state at a particular migration point.

They should be accessed through the migration's `apps` registry.

```python
User = apps.get_model("users", "User")
```

Avoid importing the current model directly inside a migration because the model may change in the future.

9. Squashing Migrations

Migration squashing combines multiple migration files into a smaller set of migrations.

```bash
python manage.py squashmigrations app_name 0001 0005
```

Squashing can reduce the number of migration files that Django needs to process.

Do not delete replaced migration files until the squashed migration has been adopted safely across the relevant environments.

10. Migration Best Practices

- Commit migration files to version control.
- Run migrations in development and test environments before production.
- Review generated migration files.
- Back up important production data.
- Avoid manually editing applied migrations unless you understand the consequences.
- Use data migrations for controlled data transformations.

Part 2: Django File Management

1. FileField and ImageField

Django provides `FileField` and `ImageField` to manage uploaded files.

```python
from django.db import models


class Document(models.Model):
    title = models.CharField(max_length=100)
    file = models.FileField(upload_to="documents/")
    image = models.ImageField(
        upload_to="images/",
        blank=True,
        null=True,
    )
```

`FileField` stores a reference to a file, while `ImageField` also performs image validation.

To use `ImageField`, install Pillow:

```bash
pip install Pillow
```

2. Configure Media Files

In `settings.py`:

```python
MEDIA_URL = "/media/"
MEDIA_ROOT = BASE_DIR / "media"
```

During development, media files can be served through Django's development server configuration.

Production media files should normally be served by a web server or object storage service.

3. File Upload Form

```python
from django import forms


class DocumentForm(forms.Form):
    title = forms.CharField(max_length=100)
    file = forms.FileField()
```

In the HTML template:

```html
<form method="post" enctype="multipart/form-data">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Upload</button>
</form>
```

The `multipart/form-data` encoding is required for uploading files.

4. Handling Uploaded Files

```python
from django.shortcuts import render, redirect
from .forms import DocumentForm
from .models import Document


def upload_document(request):
    if request.method == "POST":
        form = DocumentForm(request.POST, request.FILES)

        if form.is_valid():
            Document.objects.create(
                title=form.cleaned_data["title"],
                file=form.cleaned_data["file"],
            )
```

```
            return redirect("success")

    else:
        form = DocumentForm()

    return render(
        request,
        "upload.html",
        {"form": form},
    )
```

5. File Storage

Django uses storage backends to determine how and where files are stored.

Common storage options include:

- Local filesystem storage.
- Cloud object storage.
- Custom storage backends.

A model field generally stores the file's name or path in the database. The file contents are handled by the storage backend.

6. File Security

- Validate file size and file type.
- Do not trust the original filename.
- Avoid executing uploaded files.
- Keep private uploads outside publicly accessible locations.
- Configure appropriate permissions.
- Use malware scanning where appropriate.
- Consider storage quotas and upload limits.

Part 3: Testing in Django

1. What Is Testing?

Testing checks whether an application behaves as expected.

Django provides a testing framework built on Python's `unittest`.

Tests are commonly placed in `tests.py` or a `tests/` package.

2. Writing a Simple Test

Suppose we have a model:

```python
from django.db import models


class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.DecimalField(
        max_digits=10,
        decimal_places=2,
    )
```

A test can verify that a product is created correctly:

```python
from django.test import TestCase
from .models import Product


class ProductTests(TestCase):

    def test_create_product(self):
        product = Product.objects.create(
            name="Keyboard",
            price="1200.00",
        )

        self.assertEqual(product.name, "Keyboard")
        self.assertEqual(str(product.price), "1200.00")
```

3. Run Tests

Run all tests:

```bash
python manage.py test
```

Run tests for one app:

```bash
python manage.py test products
```

Run a particular test class:

```bash
python manage.py test products.tests.ProductTests
```

4. Testing Views

Django's test client simulates HTTP requests without requiring a real browser.

```python
from django.test import TestCase
from django.urls import reverse


class HomePageTests(TestCase):

    def test_home_page(self):
        response = self.client.get(reverse("home"))

        self.assertEqual(response.status_code, 200)
```

5. Testing APIs

````
For Django REST Framework:

```python
from rest_framework.test import APITestCase
from django.urls import reverse


class ProductAPITests(APITestCase):

    def test_product_list(self):
        response = self.client.get(
            reverse("product-list")
        )

        self.assertEqual(response.status_code, 200)
```

The URL name must exist in the project's URL configuration.

## 6. Types of Tests

- **Unit tests:** Test small units of code.
- **Integration tests:** Test how multiple components work together.
- **API tests:** Test HTTP endpoints, status codes, and response data.
- **Authentication tests:** Test login, permissions, and access control.
- **Regression tests:** Ensure previously fixed bugs do not return.

## 7. Testing Best Practices

- Test successful and unsuccessful cases.
- Test permissions and authentication.
- Test input validation.
- Test edge cases.
- Use descriptive test names.
- Avoid relying on production data.
- Run tests before deploying changes.

---

# Part 4: Django User Authentication
````

1. What Is Authentication?

Authentication verifies the identity of a user.

Django provides a built-in authentication system with users, passwords, groups, permissions, and sessions.

Authentication is different from authorization:

- **Authentication:** Who are you?
- **Authorization:** What are you allowed to do?

2. Configure Authentication Apps

Django projects created with the standard configuration usually include:

```python
INSTALLED_APPS = [
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
]
```

Apply migrations:

```bash
python manage.py migrate
```

Create an administrator:

```bash
python manage.py createsuperuser
```

3. Create a User

```python
from django.contrib.auth.models import User
```

```
user = User.objects.create_user(
    username="faruk",
    email="faruk@example.com",
    password="StrongPassword123!",
)
```

Use `create_user()` rather than assigning a raw password directly.

4. Authenticate a User

```python
from django.contrib.auth import authenticate

user = authenticate(
    username="faruk",
    password="StrongPassword123!",
)

if user is not None:
    print("Authentication successful")
else:
    print("Invalid credentials")
```

5. Login and Logout

```python
from django.contrib.auth import authenticate, login, logout
from django.shortcuts import redirect


def login_view(request):
    if request.method == "POST":
        user = authenticate(
            request,
            username=request.POST.get("username"),
            password=request.POST.get("password"),
        )

        if user is not None:
```

```
            login(request, user)
            return redirect("home")

    return redirect("login")


def logout_view(request):
    logout(request)
    return redirect("login")
```

In production, use Django forms or serializers to validate credentials and handle errors properly. Protect state-changing requests against CSRF.

6. Protect Views

```python
from django.contrib.auth.decorators import login_required


@login_required
def dashboard(request):
    return render(request, "dashboard.html")
```

Only authenticated users can access the protected view.

7. Custom User Models

A custom user model is useful when the application requires additional user fields or different authentication behavior.

Configure `AUTH_USER_MODEL` before creating the initial migrations for the project whenever possible.

```python
AUTH_USER_MODEL = "accounts.CustomUser"
```

Use `settings.AUTH_USER_MODEL` in model relationships and

`get_user_model()` when retrieving the active user model in application code.

Part 5: Django Cache Framework

1. What Is Caching?

Caching stores frequently used data so it can be retrieved faster instead of being recalculated or fetched repeatedly.

Benefits include:

- Faster responses.
- Reduced database queries.
- Lower server workload.
- Better scalability.

2. Configure a Cache

Example using a local-memory cache:

```python
CACHES = {
    "default": {
        "BACKEND": (
            "django.core.cache.backends.locmem.LocMemCache"
        ),
        "LOCATION": "unique-cache-name",
    }
}
```

Local-memory caching is useful for development, but each process maintains its own cache. Shared production deployments often benefit from Redis or another shared cache backend.

3. Low-Level Cache API

```python
```

```
from django.core.cache import cache

cache.set("username", "faruk", timeout=300)

value = cache.get("username")

cache.delete("username")
```

Here, `timeout=300` means the entry expires after 300 seconds.

4. Cache a Function Result

```python
from django.core.cache import cache


def get_expensive_data():
    key = "expensive_data"

    data = cache.get(key)

    if data is None:
        data = {"message": "Calculated result"}
        cache.set(key, data, timeout=600)

    return data
```

For concurrent requests, consider whether duplicate calculations are acceptable and whether cache invalidation is correct.

5. Per-View Caching

```python
from django.views.decorators.cache import cache_page


@cache_page(60 * 15)
def product_list(request):
    ...
```

```

This caches the view response for 15 minutes.

Be careful when caching user-specific or permission-dependent responses.
Ensure cached content cannot leak between users.

## 6. Template Fragment Caching

```django
{% load cache %}

{% cache 300 sidebar %}
    <div>
        Frequently displayed sidebar content
    </div>
{% endcache %}
```

The fragment is cached for 300 seconds.

7. Cache Invalidation

Cache invalidation means removing or updating cached data when the
underlying data changes.

```python
from django.core.cache import cache

cache.delete("product_list")
```

A cache strategy should define:

- Cache keys.
- Expiration times.
- Invalidation rules.
- User-specific variation.
- Behavior when the cache is unavailable.

```

Part 6: Conditional View Processing

1. What Is Conditional View Processing?

Conditional view processing lets Django evaluate request conditions before performing the full work of a view.

It can help avoid unnecessary processing when a client already has an up-to-date representation of a resource.

HTTP conditional requests commonly use headers such as:

- `If-Modified-Since`
- `If-None-Match`
- `Last-Modified`
- `ETag`

2. The `condition` Decorator

Django provides a `condition` decorator that can use functions to calculate ETags and last-modified timestamps.

```python
from django.views.decorators.http import condition


def product_etag(request, product_id):
    return '"product-version-1"'


@condition(etag_func=product_etag)
def product_detail(request, product_id):
    ...
```

The example uses a fixed ETag only for demonstration. In a real application, derive the ETag from the resource's actual version or representation.

When a client's conditional request indicates that the resource has not

````
changed, Django may return `304 Not Modified`.

## 3. Why Use Conditional Requests?

- Reduce unnecessary response-body transfers.
- Save bandwidth.
- Improve caching behavior.
- Reduce repeated work for suitable resources.

Conditional requests should be designed carefully when responses vary by user, permissions, or content negotiation.

---

# Part 7: Composite Primary Keys

## 1. What Is a Composite Primary Key?

A composite primary key consists of multiple fields that jointly identify a database row.

For example, a membership table might identify a record by both `student_id` and `course_id`.

Conceptually:

```text
student_id + course_id = unique row identity
```

## 2. Django Support

Support for composite primary keys depends on the Django version. Django 5.2 introduced `CompositePrimaryKey`.

Example:

```python
from django.db import models
````
````

```
class Membership(models.Model):
    student_id = models.IntegerField()
    course_id = models.IntegerField()

    pk = models.CompositePrimaryKey(
        "student_id",
        "course_id",
    )
```

The combination of `student_id` and `course_id` forms the primary key.

3. Important Limitations

Composite primary keys have limitations in some Django features, particularly certain relationships and third-party integrations.

Before adopting them, check the documentation for the Django version being used and verify compatibility with forms, admin workflows, and related models.

For many applications, a conventional single-field primary key plus a database `UniqueConstraint` may be simpler.

Part 8: Cryptographic Signing

1. What Is Cryptographic Signing?

Cryptographic signing allows an application to detect whether data has been modified.

A signature is generated using a secret key. When the signed data is received again, Django can verify its authenticity.

Signing does not encrypt data. Signed content may still be readable.

2. Sign Data

```python
```

````
from django.core import signing

signed_value = signing.dumps({
    "user_id": 10,
    "role": "editor",
})
```

## 3. Verify and Decode Data

```python
from django.core import signing

try:
    data = signing.loads(
        signed_value,
        max_age=3600,
    )
except signing.BadSignature:
    data = None
```

The `max_age` argument limits how long the signed value is accepted.

## 4. Security Best Practices

- Keep `SECRET_KEY` private.
- Do not commit production secrets to version control.
- Use environment variables or a secrets manager.
- Rotate secrets carefully.
- Configure `SECRET_KEY_FALLBACKS` when appropriate for a planned key rotation.
- Remember that signing provides integrity, not confidentiality.

---

# Part 9: Sending Email

## 1. Configure Email

Example SMTP configuration:
````
````

```python
EMAIL_BACKEND = (
    "django.core.mail.backends.smtp.EmailBackend"
)

EMAIL_HOST = "smtp.example.com"
EMAIL_PORT = 587
EMAIL_USE_TLS = True

EMAIL_HOST_USER = "your-email@example.com"
EMAIL_HOST_PASSWORD = "your-email-password"

DEFAULT_FROM_EMAIL = "your-email@example.com"
```

Keep credentials in environment variables or a secrets manager instead of hardcoding them.

2. Send a Simple Email

```python
from django.core.mail import send_mail

send_mail(
    subject="Welcome",
    message="Welcome to our website.",
    from_email=None,
    recipient_list=["user@example.com"],
    fail_silently=False,
)
```

If `from_email` is `None`, Django uses `DEFAULT_FROM_EMAIL`.

3. Send Multiple Emails

```python
from django.core.mail import send_mass_mail

messages = (
```

```
    (
        "First Subject",
        "First message",
        None,
        ["first@example.com"],
    ),
    (
        "Second Subject",
        "Second message",
        None,
        ["second@example.com"],
    ),
)

send_mass_mail(messages, fail_silently=False)
```

4. EmailMessage

`EmailMessage` supports more advanced email features, including attachments and custom headers.

```python
from django.core.mail import EmailMessage

email = EmailMessage(
    subject="Report",
    body="Please find the report attached.",
    to=["user@example.com"],
)

email.attach_file("/path/to/report.pdf")
email.send()
```

5. Development Email Backend

To print email messages in the terminal instead of sending them:

```python
EMAIL_BACKEND = (
```

```
    "django.core.mail.backends.console.EmailBackend"
)
```

This is useful for development and testing.

Part 10: Internationalization and Localization

1. Internationalization (i18n)

Internationalization prepares an application to support multiple languages and regional formats.

2. Localization (l10n)

Localization adapts an application to a particular language, region, date format, or number format.

3. Translation Strings

```python
from django.utils.translation import gettext_lazy as _

title = _("Welcome")
```

In templates:

```django
{% load i18n %}

<h1>{% translate "Welcome" %}</h1>
```

4. Language Settings

```python
LANGUAGE_CODE = "en-us"
```

````
USE_I18N = True
USE_TZ = True
```

Language selection and available languages can be configured according to application requirements.

## 5. Generate Translation Files

```bash
django-admin makemessages -l fr
django-admin compilemessages
```

The first command extracts translatable strings for French. The second compiles the translation files.

---

# Part 11: Logging

## 1. What Is Logging?

Logging records information about application behavior, errors, and important events.

It helps developers troubleshoot problems and monitor applications.

## 2. Logging Levels

| Level | Purpose |
|---|---|
| DEBUG | Detailed diagnostic information |
| INFO | General informational messages |
| WARNING | Unexpected situations that may need attention |
| ERROR | An operation failed |
| CRITICAL | A serious failure |

## 3. Basic Logging Example

```python
````

```
import logging

logger = logging.getLogger(__name__)


def process_order():
    logger.info("Order processing started")

    try:
        # Perform the operation
        pass
    except Exception:
        logger.exception("Order processing failed")
```

`logger.exception()` includes exception information when called inside an exception handler.

4. Logging Configuration

Django uses Python's logging framework and supports configuration through the `LOGGING` setting.

A production configuration may direct logs to files, standard output, or a centralized logging system.

Avoid logging passwords, secret keys, access tokens, or sensitive personal data.

Part 12: Pagination

1. What Is Pagination?

Pagination divides a large set of results into smaller pages.

It improves usability and can reduce the amount of data processed and returned at once.

2. Using Paginator

```python
from django.core.paginator import Paginator
from django.shortcuts import render
from .models import Product


def product_list(request):
    products = Product.objects.order_by("id")

    paginator = Paginator(products, 10)

    page_number = request.GET.get("page")
    page_obj = paginator.get_page(page_number)

    return render(
        request,
        "products/list.html",
        {"page_obj": page_obj},
    )
```

This example displays up to ten products per page.

3. Pagination Template

```django
{% for product in page_obj %}
    <p>{{ product.name }}</p>
{% empty %}
    <p>No products found.</p>
{% endfor %}

{% if page_obj.has_previous %}
    <a href="?page={{ page_obj.previous_page_number }}">
        Previous
    </a>
{% endif %}

<span>
    Page {{ page_obj.number }} of {{ page_obj.paginator.num_pages }}
```

```
    </span>

    {% if page_obj.has_next %}
        <a href="?page={{ page_obj.next_page_number }}">
            Next
        </a>
    {% endif %}
    ```

    For very large datasets, consider database-level pagination strategies and stable ordering.

    # Part 13: Django Security

    ## 1. Cross-Site Scripting (XSS)

    XSS occurs when malicious scripts are injected into pages viewed by other users.

    Django templates automatically escape many variable values by default.

    ```django
    <p>{{ user_input }}</p>
    ```

    Avoid using `safe` or disabling autoescaping on untrusted input.

    ## 2. Cross-Site Request Forgery (CSRF)

    CSRF tricks an authenticated browser into sending an unwanted request.

    For a standard Django POST form:

    ```django
    <form method="post">
        {% csrf_token %}
        <button type="submit">Submit</button>
    </form>
    ```
```
```

````
Keep CSRF protection enabled for applicable browser-based requests.

## 3. SQL Injection

SQL injection occurs when untrusted input is incorporated into SQL in an
unsafe way.

Prefer Django ORM queries:

```python
Product.objects.filter(name=user_input)
```

For raw SQL, use parameterized queries instead of string concatenation.

## 4. Clickjacking

Clickjacking attempts to trick users into interacting with a page
embedded in another page.

Django provides clickjacking protection through middleware and related
response headers.

## 5. HTTPS and SSL

Use HTTPS in production to protect data in transit.

Common settings include:

```python
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True
```

Configure HTTPS redirects and HSTS only when the deployment's HTTPS
setup is correct.

## 6. Host Header Validation

Configure allowed hosts:
````

```python
ALLOWED_HOSTS = [
    "example.com",
    "www.example.com",
]
```

Do not use an unrestricted host configuration in production.

7. Session Security

Use secure cookie settings, appropriate session expiration, and HTTPS.

Do not store passwords or other secrets directly in session data.

8. User Upload Security

Uploaded files are untrusted input.

Validate file types and sizes, prevent executable uploads, and store sensitive files securely.

9. Content Security Policy (CSP)

A Content Security Policy helps restrict the sources from which browsers may load resources.

Choose a policy appropriate to the application's scripts, styles, images, and other resources. A restrictive policy should be tested before enforcement.

10. Security Checklist

- Keep Django and dependencies updated.
- Set `DEBUG = False` in production.
- Configure `ALLOWED_HOSTS`.
- Use HTTPS.
- Protect secrets.
- Validate user input.
- Use CSRF protection.

- Avoid unsafe raw SQL.
- Set appropriate permissions.
- Review production security settings with Django's deployment checklist.

Part 14: Performance and Optimization

1. Database Query Optimization

Use `select_related()` for suitable foreign-key and one-to-one relationships.

```python
orders = Order.objects.select_related("customer")
```

Use `prefetch_related()` for many-to-many and reverse relationships where appropriate.

```python
products = Product.objects.prefetch_related("categories")
```

These methods can reduce unnecessary database queries.

2. Avoid N+1 Queries

An N+1 query problem occurs when an application executes one query for a collection and then an additional query for each item.

Use query inspection and tools such as Django Debug Toolbar in development to identify this issue.

3. Use `only()` and `defer()` Carefully

```python
products = Product.objects.only("id", "name")
```

````
These methods can reduce the fields initially loaded, but accessing
deferred fields may cause additional queries.

## 4. Caching

Cache expensive computations or frequently accessed data when doing so
is correct for the data's freshness and access rules.

## 5. Database Indexes

Indexes can improve lookup performance.

```python
class Product(models.Model):
    name = models.CharField(max_length=100, db_index=True)
```

Indexes also add storage and write overhead. Add them based on query
patterns and database measurements.

## 6. Lazy Evaluation

Django QuerySets are generally lazy: constructing a QuerySet does not
immediately execute its database query.

The query is usually executed when the results are evaluated, for
example by iterating over the QuerySet or converting it to a list.

## 7. General Optimization Strategy

1. Measure the performance problem.
2. Identify the bottleneck.
3. Inspect database queries.
4. Optimize the relevant code.
5. Measure again.
6. Add regression tests where appropriate.

Avoid optimizing code based only on assumptions.

---
````

````
# Part 15: Serializing Django Objects

## 1. What Is Serialization?

Serialization converts data into a format that can be stored or
transferred.

Django provides serialization utilities for model data.

## 2. Serialize Model Data

```python
from django.core import serializers
from .models import Product

data = serializers.serialize(
    "json",
    Product.objects.all(),
)
```

This produces JSON representing the selected model records.

## 3. Deserialize Data

```python
from django.core import serializers

objects = serializers.deserialize("json", data)

for obj in objects:
    print(obj.object)
```

Deserialization produces objects that can be inspected and saved when
appropriate.

## 4. Natural Keys

Natural keys use meaningful field values to identify related objects
rather than relying only on database-generated primary keys.
````

They can be useful when moving fixture data between databases, provided the relevant models implement appropriate natural-key behavior.

5. Serialization for APIs

Django's core serializers are not the same as Django REST Framework serializers.

DRF serializers are commonly used to validate incoming API data and convert model instances into API response representations.

Part 16: Django Settings

1. What Are Settings?

Django settings define how a project behaves, including installed apps, middleware, databases, templates, security, and static files.

2. Common Settings

```python
DEBUG = True

ALLOWED_HOSTS = []

INSTALLED_APPS = []

MIDDLEWARE = []

ROOT_URLCONF = "project.urls"

WSGI_APPLICATION = "project.wsgi.application"

ASGI_APPLICATION = "project.asgi.application"
```

The actual project configuration may contain additional settings.

3. Environment Variables

Sensitive configuration should not be hardcoded in source code.

```python
import os

SECRET_KEY = os.environ["DJANGO_SECRET_KEY"]

DEBUG = os.environ.get("DJANGO_DEBUG", "False") == "True"
```

Configure environment variables securely for each environment.

4. Access Settings in Python

```python
from django.conf import settings

print(settings.DEBUG)
```

Use `django.conf.settings` rather than importing project settings directly.

5. Separate Environments

Development, testing, and production environments often require different settings.

For example:

- Development: debug tools and local services.
- Testing: test databases and test-specific configuration.
- Production: secure cookies, HTTPS, production databases, and protected secrets.

Part 17: Django Signals

````
## 1. What Are Signals?

Signals allow parts of an application to be notified when particular
events occur.

They can help decouple components, but they can also make program flow
harder to understand if overused.

## 2. Example: `post_save`

```python
from django.db.models.signals import post_save
from django.dispatch import receiver
from django.contrib.auth import get_user_model

User = get_user_model()


@receiver(post_save, sender=User)
def user_saved(sender, instance, created, **kwargs):
    if created:
        print("A new user was created")
```

The receiver runs after a `User` instance is saved.

## 3. Register Signals

A common pattern is to define signal handlers in `signals.py` and import
them from the app's `AppConfig.ready()` method.

```python
from django.apps import AppConfig


class AccountsConfig(AppConfig):
    name = "accounts"

    def ready(self):
        from . import signals
```
````

Ensure the app configuration is loaded correctly.

4. Common Signals

- `pre_save`
- `post_save`
- `pre_delete`
- `post_delete`
- `m2m_changed`
- `request_started`
- `request_finished`

5. Signal Best Practices

- Avoid unnecessary signals.
- Keep handlers small.
- Prevent recursive save operations.
- Do not assume a signal automatically creates a reliable background job.
- Use explicit service functions when they make business logic easier to understand.

Part 18: Django System Check Framework

1. What Is the System Check Framework?

The system check framework identifies common configuration and application problems.

Run checks with:

```bash
python manage.py check
```

For deployment-related checks:

```bash
```

```
python manage.py check --deploy
```

Deployment checks should be run with the appropriate production settings.

2. Custom System Checks

Django allows applications to define custom checks for project-specific configuration rules.

Custom checks can help identify missing configuration and other issues before deployment.

Part 19: External Django Packages

1. Django Localflavor

`django-localflavor` provides country-specific fields and validators for data such as postal codes and regional identifiers.

Install it with:

```bash
pip install django-localflavor
```

Check the package documentation for the specific country and field required.

2. Django Formtools

Django Formtools provides additional form utilities, including multi-step form workflows.

Install it with:

```bash
pip install django-formtools
```

```

It can be useful when a long form should be divided into several steps.

## 3. Package Best Practices

- Read the official documentation.
- Check compatibility with your Django version.
- Keep dependencies updated.
- Review maintenance and security status.
- Avoid installing packages for features Django already provides adequately.

---

# Part 20: Django Asynchronous Support

## 1. What Is Async?

Asynchronous programming allows a program to handle waiting operations without blocking the execution flow in the same way as synchronous code.

It is particularly useful for suitable I/O-bound workloads.

## 2. Async View

```python
import asyncio
from django.http import JsonResponse


async def hello(request):
    await asyncio.sleep(1)

    return JsonResponse({
        "message": "Hello from an async view"
    })
```

The example demonstrates an asynchronous view. In a real application, use async-compatible libraries for asynchronous network or database
```

operations.

3. Async and Sync Compatibility

Django supports both synchronous and asynchronous views.

When calling synchronous-only code from asynchronous code, use an appropriate adapter such as `sync_to_async()` when necessary.

```python
from asgiref.sync import sync_to_async


def get_value():
    return 100


async def get_value_async():
    value = await sync_to_async(get_value)()
    return value
```

4. Async Safety

Some Django operations are synchronous-only or have specific async limitations.

Do not call blocking synchronous code directly inside an async view if it can block the event loop. Check the documentation for the relevant Django version and component.

5. ASGI

ASGI supports asynchronous application interfaces and is commonly used when deploying Django applications that need async capabilities.

WSGI remains suitable for many synchronous applications.

Part 21: Django Tasks Framework

1. What Is Background Task Processing?

Background tasks allow work to be scheduled or executed separately from the main request-response flow.

Examples include:

- Sending emails.
- Processing uploaded files.
- Generating reports.
- Performing time-consuming operations.
- Processing queued work.

2. Django Tasks Framework

Django 6.0 introduced the Tasks framework as an interface for defining and queuing background work.

The framework provides task definitions and queuing abstractions; executing tasks still requires an appropriate task backend and worker setup.

3. Example Task Definition

The exact configuration and backend depend on the Django version and the selected task backend.

A conceptual task might look like this:

```python
def generate_report(report_id):
    # Load report data.
    # Generate the report.
    # Store the result.
    pass
```

In a real Django Tasks implementation, define and enqueue the task according to the documented API for the Django version in use.

```
## 4. Important Background Task Practices

- Keep tasks idempotent where possible.
- Handle exceptions.
- Log failures.
- Configure retries appropriately.
- Avoid passing large or sensitive objects unnecessarily.
- Monitor queue processing.
- Understand whether tasks may execute more than once.
- Do not assume that enqueuing a task means it has already completed.

---

# Part 22: Common Django Commands

| Command | Purpose |
|---|---|
| `django-admin startproject project_name` | Create a project |
| `python manage.py startapp app_name` | Create an app |
| `python manage.py runserver` | Start the development server |
| `python manage.py makemigrations` | Create migration files |
| `python manage.py migrate` | Apply migrations |
| `python manage.py showmigrations` | Display migration status |
| `python manage.py createsuperuser` | Create an administrator |
| `python manage.py changepassword username` | Change a user's password
|
| `python manage.py shell` | Open the Django shell |
| `python manage.py test` | Run tests |
| `python manage.py check` | Run system checks |
| `python manage.py check --deploy` | Run deployment checks |
| `python manage.py collectstatic` | Collect static files |
| `python manage.py sqlmigrate app_name 0001` | Display migration SQL |

---

# Part 23: Production Deployment Checklist

Before deploying a Django application:

- [ ] Set `DEBUG = False`.
- [ ] Configure `ALLOWED_HOSTS`.
```

```
- [ ] Protect `SECRET_KEY` and other credentials.
- [ ] Configure a production database.
- [ ] Apply database migrations.
- [ ] Configure static files.
- [ ] Configure media storage.
- [ ] Use HTTPS.
- [ ] Configure secure session and CSRF cookies.
- [ ] Run `python manage.py check --deploy`.
- [ ] Run automated tests.
- [ ] Configure logging and error monitoring.
- [ ] Back up important data.
- [ ] Review permissions and upload security.
- [ ] Confirm that environment variables are configured correctly.

---

# Part 24: Django Interview Quick Revision

**Q1. What are migrations?**

Migrations track and apply changes to the database schema based on model changes.

**Q2. What is the difference between `makemigrations` and `migrate`?**

`makemigrations` creates migration files. `migrate` applies migrations to the database.

**Q3. What is the difference between authentication and authorization?**

Authentication verifies identity. Authorization determines what an authenticated user can access.

**Q4. What is caching?**

Caching stores frequently used data or responses so they can be accessed faster.

**Q5. What is pagination?**

Pagination divides a large result set into smaller pages.
```

Q6. What is a signal?

A signal allows one part of an application to notify other registered handlers about an event.

Q7. What is a QuerySet?

A QuerySet represents a collection of database queries and their results. It is generally evaluated lazily.

Q8. What is the N+1 query problem?

It occurs when one query retrieves a collection and additional queries are executed for individual records.

Q9. What is the difference between `select_related()` and `prefetch_related()`?

`select_related()` uses SQL joins for suitable single-valued relationships. `prefetch_related()` retrieves related objects separately and joins them in Python.

Q10. What is Django middleware?

Middleware is a component that processes requests and responses globally or across selected request flows.

Q11. What is CSRF protection?

CSRF protection helps prevent unauthorized state-changing requests initiated through an authenticated browser session.

Q12. Does cryptographic signing encrypt data?

No. Signing helps detect tampering but does not hide the data.

Q13. What is an async view?

An async view is defined with `async def` and can await compatible asynchronous operations.

Q14. What is the purpose of Django's testing framework?

It verifies application behavior and helps prevent regressions.

Q15. What is the purpose of Django's Tasks framework?

It provides abstractions for defining and queuing background work. A task backend and worker setup are still needed to execute queued tasks.

Recommended Learning Order

1. Models and ORM
2. Migrations
3. Views and URL routing
4. Templates and forms
5. Authentication and permissions
6. File uploads and storage
7. Testing
8. REST APIs with Django REST Framework
9. Pagination and filtering
10. Security
11. Logging and error handling
12. Query optimization and caching
13. Email and background tasks
14. Async support
15. Deployment and production configuration